



FileFlow Studio

User Manual and Reference Guide

FileFlow Studio — Motor Visual de Automatización DAG en .NET 9



User Manual and Node Reference Guide

FileFlow Studio v2.0

Node-Based File Automation and Batch Processing Platform built with .NET 9 and C# 13



Table of Contents

1. Introduction and Design Philosophy
2. Fundamental Editor Concepts
 - Node Canvas (Nodify)
 - The File Context (`FileItemContext`)
 - Nested Sub-Flows and Multi-Level Macros (Breadcrumbs)
 - Real-Time Telemetry Badges
3. Execution Modes and Data Safety
 - Normal Execution
 - Virtual Simulation Mode ("Dry Run")
 - Rollback System and Windows Recycle Bin
 - Interactive Debugging with Breakpoints
4. Token Engine and Dynamic Variables
 - Token Syntax
 - Built-In Providers and Functions
5. Exhaustive Node Catalog
 - Category 1: FileSystem (Disk I/O)
 - Category 2: Logic (Flow Control)
 - Category 3: Hashing (Integrity and Deduplication)

- Category 4: Archives (Compression and Extraction)
- Category 5: Images & Media (Multimedia and EXIF)
- Category 6: Scripting (Custom C# & JavaScript)
- Category 7: Integrations (CLI and Webhooks)

6. Example Flows and Built-In Templates

1. Introduction and Design Philosophy

FileFlow Studio is a visual batch file automation platform designed to build interactive file processing pipelines, inspired by modern node-based tools such as *n8n*, *ComfyUI*, and *Node-RED*.

Key Features:

- **Plugin-Based Modularity:** Every functional area lives in an isolated, zero-touch plugin (`FileFlow.Plugin.*`).
 - **Guaranteed Safety:** Non-destructive processing by default, virtual simulation (*Dry Run*), and safe deletion via Windows Recycle Bin.
 - **Asynchronous .NET 9 Performance:** High-throughput reactive pipelines powered by `System.Threading.Channels` and in-memory SQLite telemetry (>82,000 logs/sec).
 - **Nested Sub-Flows:** Encapsulate complex sub-graphs into reusable macro nodes with breadcrumb navigation (`Root > Macro A > Macro B`).
 - **Dynamic Internationalization (i18n):** Instant on-the-fly language switching (English / Spanish) across the entire UI.
-

2. Fundamental Editor Concepts

Node Canvas (Nodify)

The canvas lets you drag and drop nodes from the left **Toolbox**. Each node features:

- **Input Ports:** Located on the left edge. Receive incoming files or trigger signals.
- **Output Ports:** Located on the right edge. Emit processed files or conditional branch items.
- **Status LED:** Visual glowing indicator reflecting the current state (`Idle`, `Running`, `Completed`, `Paused`, `Faulted`).
- **Breakpoint Toggle:** Click the top-left circle to pause execution when a file reaches that specific node.
- **Header Actions Bar (`CustomActions`):** Quick-access buttons (`Method Pipeline ...`, `+ Add Variable`, `+ Add Case`, `Script Editor ...`) directly on the card.

The File Context (`FileItemContext`)

A lightweight, immutable/transmutable record traveling through DAG edges containing:

- `CurrentPath` : The current on-disk physical or virtual path of the file at this step.
- `OriginalPath` : The immutable entry path where the file was initially ingested.
- `FileSizeBytes` : Exact file size in bytes.
- `Metadata` : Key-value dictionary enriched by upstream nodes (`Exif:` , `Hash:` , `Cli:` , `Doc:` , etc.).
- `Tags` : A `HashSet` collection of labels and operational flags.
- `ExecutionLog` : Trace history of all transformations applied to this specific item.

Nested Sub-Flows and Multi-Level Macros (Breadcrumbs)

Sub-flows allow encapsulating complex graph sections into a single compound node:

1. Double-click the sub-flow node or click **"Open Sub-flow"**.
2. The canvas displays the inner graph and the top breadcrumb navigation bar: `Root Workflow > Sub-flow A > Sub-flow B`.
3. Clicking on any previous level automatically saves changes and returns to the parent canvas.

Real-Time Telemetry Badges

During pipeline execution, every connection wire updates dynamically with live item counts (e.g.,  1,250 items), helping you pinpoint bottlenecks and observe data routing in real time.

3. Execution Modes and Data Safety

Normal Execution

Click the green **"▶ Run Workflow"** button in the top toolbar. The engine processes all files concurrently according to the configured degree of parallelism.

Virtual Simulation Mode ("Dry Run")

1. Check the **"Dry Run"** toggle or click **"Virtual Simulation"**.
2. The engine executes the complete graph resolving names, paths, hashes, and logic **without writing, moving, or deleting any physical files on disk**.
3. The bottom console outputs an exact list of all planned actions (`PlannedAction`).

Rollback System and Windows Recycle Bin

- **Safe Deletion:** All deletion operations use the native Windows Shell API (`SHFileOperation`), moving files to the Recycle Bin rather than permanently deleting them.
- **Undo / Rollback (Ctrl+Z):** If you wish to revert changes after running a flow, click the **"↶ Undo / Rollback"** button. The engine undoes renames and moves in LIFO order (last executed, first undone).

Interactive Debugging with Breakpoints

- Click **"🐛 Debug Workflow"**.

- Execution automatically pauses when an item reaches a node with an active breakpoint or when an exception occurs.
 - Use "**Step Over (F10)**" to advance one node at a time and inspect metadata diffs in the **Node Inspector Panel**.
-

4. Token Engine and Dynamic Variables

Any text input, output directory, or file name template supports dynamic variables enclosed in `{ ... }`.

Syntax:

`{Domain:Key:Modifier}` or `{Variable}`

Available Token Catalog:

Token	Output Example	Description
<code>----</code>	<code>----</code>	<code>----</code>
<code>{FileName}</code>	<code>document.pdf</code>	File name with extension
<code>{FileNameNoExt}</code>	<code>document</code>	File name without extension
<code>{Ext}</code> or <code>{Extension}</code>	<code>pdf</code>	File extension without leading dot
<code>{CurrentDir}</code>	<code>C:\MyFiles</code>	Current directory path
<code>{ParentDir}</code>	<code>MyFiles</code>	Name of the immediate parent folder
<code>{CreationDate:yyyyMMdd}</code>	<code>20260902</code>	Formatted creation date
<code>{ModifiedDate:yyyy-MM-dd}</code>	<code>2026-09-02</code>	Formatted last modification date
<code>{Now:yyyyMMdd_HH:mm:ss}</code>	<code>20260902_143000</code>	Current system timestamp
<code>{FileSize:MB}</code> or <code>{SizeMB}</code>	<code>14.50</code>	Size in Megabytes
<code>{FileSize:KB}</code> or <code>{SizeKB}</code>	<code>14848.0</code>	Size in Kilobytes
<code>{Hash:SHA256}</code>	<code>e3b0c44298fc1c ...</code>	Full calculated SHA-256 hash
<code>{Hash:SHA256:8}</code>	<code>e3b0c442</code>	SHA-256 hash truncated to 8 characters
<code>{Exif:CameraModel}</code>	<code>Sony ILCE-7M4</code>	Camera model from EXIF metadata
<code>{Exif:Make}</code>	<code>SONY</code>	Camera manufacturer
<code>{Doc:PageCount}</code>	<code>18</code>	Total page count (PDF/Office docs)
<code>{Media:Duration}</code>	<code>00:45:12</code>	Video or audio duration
<code>{Env:USERPROFILE}</code>	<code>C:\Users\username</code>	OS environment variable
<code>{Index:D4}</code>	<code>0001</code> , <code>0002</code>	Batch sequential zero-padded counter

5. Exhaustive Node Catalog

Category 1: FileSystem (Disk I/O)

1. `FolderSourceNode` (Folder Source)

- **Purpose:** Scans a directory and emits discovered files into the pipeline.

- **Ports:**
- *Outputs:* `FileOut` (`FileItemContext`), `Completed` (End signal).
- **Parameters:**
- `SourceFolder` : Root folder path to scan (supports tokens).
- `SearchPattern` : File filter (e.g., `.` , `.jpg; .png`).
- `IncludeSubdirectories` : `true` for recursive deep scanning.

2. `AdvancedRenamerNode` (Advanced Multi-Method Renamer)

- **Purpose:** Batch renames files using an extensible pipeline of **9 sequential method steps**, emulating professional renaming suites.
- **Ports:**
- *Inputs:* `In` (`FileItemContext`)
- *Outputs:* `Out` (Success), `Skipped` (Collision skipped), `Error` (I/O or validation failure)
- **Parameters & Methods:**
- `RenameMode` : `Virtual` (default: projects name into context without altering source disk file) or `DirectInPlace` .
- `CollisionStrategy` : `AutoIncrement` (`_1` , `_2`), `Overwrite` , `Skip` , `Fail` .
- `MethodSteps` : Sequential JSON pipeline of cumulative transformation methods:

1. **New Name / Template:** Complete or partial template replacement with tags (`{Tag}` or `{Tag}`).
 2. **Search & Replace:** Substring or Regex pattern matching with capture groups (`$1` , `$2`), case sensitivity, and global replace.
 3. **Insert Text:** Inserts strings at absolute or relative character positions.
 4. **Delete Characters:** Removes character ranges by count or offset.
 5. **Case Conversion:** `Lowercase` , `Uppercase` , `TitleCase` , `SentenceCase` , `CapitalizeFirst` .
 6. **Incremental Numbering:** Sequential sequence generator with initial value, step, zero-padding (e.g., `001`), and reset trigger (*DirectoryChange*, *MetadataChange*, *Never*).
 7. **Replace List (Substitutions):** Key-value mapping table for bulk dictionary replacements.
 8. **Clean, Trim & Unicode Normalization:** Whitespace trimming, double space collapse, Windows illegal character sanitization (`\` / : * ? " < > |), and Unicode normalization (`NFC` , `NFD` , `NFKC` , `NFKD`).
 9. **Normalize & Pad Numbers:** Pads embedded numbers (e.g., `1 - track.mp3` $\rightarrow$ `01 - track.mp3` , `S1E2` $\rightarrow$ `S01E02` , `Ep. 3` $\rightarrow$ `Ep. 03`).
- **Regex Studio Assistant:** Interactive tester with live capture group inspection, replacement preview, and built-in preset library.
 - **Visual Studio Window:** Live preview table with 100 loaded sample items and built-in presets (Digital Photo EXIF, ID3 Music, SEO Web, Corporate Docs).

3. FileRelocatorNode (File Relocator & Copier)

- **Purpose:** Copies or moves files to dynamically computed destinations, with optional SHA-256 integrity verification.
- **Ports:**
- *Inputs:* In
- *Outputs:* Out , Error
- **Parameters:**
- **Operation** : Copy (default, safe non-destructive) or Move .
- **DestinationDirectory** : Target directory template (e.g., {DestinationRoot}\{Year}\{Month}).
- **VerifyIntegrity** : true to verify source and target SHA-256 match.
- **CreateDirectories** : true to auto-create missing folders.

4. OriginalFileActionNode (Source File Lifecycle Action)

- **Purpose:** Centralized node for managing entry source files after downstream processing.
- **Ports:**
- *Inputs:* In
- *Outputs:* Out , Error
- **Parameters:**
- **ActionType** : Keep (default), MoveToRecycleBin (Windows Recycle Bin), MoveToQuarantine , PermanentDelete .
- **QuarantineDirectory** : Target path when action is quarantine.

5. SafeRecycleDeleteNode (Safe Delete to Recycle Bin)

- **Purpose:** Sends files to the Windows Recycle Bin using native Shell API.
- **Ports:**
- *Inputs:* In
- *Outputs:* Deleted , Error

6. EmptyDirectoryCleanerNode (Empty Directory Cleaner)

- **Purpose:** Recursively scans and cleans folders that were left empty after moving operations.
- **Ports:**
- *Inputs:* TriggerIn
- *Outputs:* Out , Error
- **Parameters:**
- **TargetDirectory** : Root folder to clean.
- **Recursive** : true for deep scan.
- **IgnoreHiddenSystemFiles** : Ignores Thumbs.db and .DS_Store .

7. **OperationReportNode** (Interactive Operations Report)

- **Purpose:** Generates visual reports tracing complete file lifecycle, grouped by source folders.
 - **Ports:**
 - *Inputs:* In
 - *Outputs:* Out , Report , Error
 - **Parameters:**
 - **ReportFormat** : HTML (interactive accordion), Markdown , Text , JSON , CSV .
 - **ReportScope** : Consolidated , PerFile , Both .
 - **GroupBy** : Directory , Flat , Extension , Status .
 - **DestinationFolder** : Target folder for the report.
 - **AutoOpenReport** : true to open in browser upon completion.
-

Category 2: Logic (Flow Control)

1. **SwitchCaseNode** (Dynamic Conditional Router)

- **Purpose:** Evaluates expressions or extensions and routes files to custom configured ports or Default .
- **Ports:**
- *Inputs:* In
- *Outputs:* Dynamic case ports (e.g., Images , Videos , Docs) + Default .

2. **ExpressionFilterNode** (Boolean Condition Filter)

- **Purpose:** Evaluates numeric or string conditions (SizeMB , Ext , CreationDate) and routes to True or False .
- **Ports:**
- *Inputs:* In
- *Outputs:* True , False
- **Parameters:**
- **Property** : Property name (SizeMB , Ext , etc.).
- **Operator** : > , < , ≥ , ≤ , == , ≠ , Contains .
- **ComparisonValue** : Target comparison value.

3. **BatchBufferNode** (Batch Accumulator)

- **Purpose:** Buffers items until reaching \$N\$ files or a maximum byte size before emitting the batch.
- **Ports:**
- *Inputs:* ItemIn , ForceFlush
- *Outputs:* ItemOut , BatchCompleted

4. ForkJoinBarrierNode (Fork/Join Synchronization Barrier)

- **Purpose:** Splices processing into multiple parallel branches and waits for all to finish before emitting `AllCompleted` .
- **Ports:**
- *Inputs:* `In` , `Branch1_Done` , `Branch2_Done`
- *Outputs:* `Fork1` , `Fork2` , `AllCompleted`

5. ThrottleDelayNode (Rate Limiter & Delay)

- **Purpose:** Introduces a configurable millisecond delay between items to prevent disk or API rate exhaustion.
 - **Ports:**
 - *Inputs:* `In`
 - *Outputs:* `Out`
-

Category 3: Hashing (Integrity & Deduplication)

1. HashCalculatorNode (Cryptographic Checksum Calculator)

- **Purpose:** Computes file hash and registers it in `Metadata["Hash: "]` and `{Hash: }` .
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Out` , `Error`
- **Parameters:**
- `Algorithm` : `SHA256` , `MD5` , `SHA512` , `SHA1` .
- `StoreInMetadataKey` : Key name (e.g., `Hash:SHA256`).

2. DeduplicationFilterNode (Hash Deduplication Filter)

- **Purpose:** Compares hashes within the current batch and splits unique items from duplicates.
 - **Ports:**
 - *Inputs:* `In`
 - *Outputs:* `Unique` , `Duplicate` , `Error`
-

Category 4: Archives (Compression & Extraction)

1. SmartUnpackNode (Smart Archive Extractor)

- **Purpose:** Unpacks ZIP, RAR, 7Z, and TAR archives with built-in multi-password dictionary support and Zip-Slip protection.

- **Ports:**
- *Inputs:* In
- *Outputs:* ExtractedFile , Done , Error
- **Parameters:**
- DestinationPath : Extraction folder.
- FlattenHierarchy : true to extract all files in a single flat directory.

2. ArchiveFilterNode (Archive Format Filter)

- **Purpose:** Filters valid archive containers from regular files.
- **Ports:**
- *Inputs:* In
- *Outputs:* ArchiveOut , NonArchiveOut

Category 5: Images & Media (Multimedia & EXIF)

1. ImageOptimizerNode (Image Optimizer & Scaler)

- **Purpose:** Resizes, compresses, and converts images to modern formats (**WebP**, **JPEG**, **PNG**) preserving aspect ratio.
- **Ports:**
- *Inputs:* In
- *Outputs:* Out , Error
- **Parameters:**
- Width : Target width in pixels, percentage, or auto ("").
- Height : Target height (default: "100%").
- TargetFormat : WebP , JPEG , PNG .
- Quality : Visual compression quality (1-100, default: 80).
- OnlyDownscale : true to prevent upscaling smaller images.

2. ExifMetadataNode (EXIF Metadata Extractor)

- **Purpose:** Extracts digital photo metadata (capture date, camera make/model, ISO, GPS coordinates) into {Exif:*} and {Date:Taken} .
- **Ports:**
- *Inputs:* In
- *Outputs:* Out , Error

3. MediaTranscoderNode (FFmpeg Multimedia Converter)

- **Purpose:** Transcodes video and audio streams using FFmpeg presets (H.264, HEVC/H.265, MP3, AAC).

- **Ports:**
 - *Inputs:* `In`
 - *Outputs:* `Out` , `Error`
-

Category 6: Scripting (Custom C# & JavaScript)

1. `CustomScriptNode` (Dynamic Dual-Engine Scripting)

- **Purpose:** Executes user scripts in **C# (Roslyn JIT with SHA256 caching)** or **JavaScript (Jint Sandboxed runtime)** with dynamic configurable ports and full metadata access.
 - **Ports:** Configurable dynamically in Script Studio (`In` , `Out` , plus any user-defined inputs/outputs).
 - **Features:**
 - `Item` / `file` : Direct typed access to file attributes, metadata, and tags.
 - `EmitAsync(port)` / `emit(port, item)` : Multi-port conditional dispatch.
 - `Resolve(template)` / `resolve(template)` : Token resolver function.
 - Interactive **Script Studio Window** with syntax highlighting, live testing, and script template library.
-

Category 7: Integrations (CLI & Webhooks)

1. `CliExecutionNode` (External CLI Command Runner)

- **Purpose:** Spawns external CLI processes (FFmpeg, Python, PowerShell) with tokenized arguments.
- **Ports:**
- *Inputs:* `In`
- *Outputs:* `Success` , `Failed`
- **Parameters:**
- `ExecutablePath` : Path to executable.
- `ArgumentsTemplate` : Dynamic command arguments.
- `TimeoutSeconds` : Timeout in seconds (default: `60`).
- `CaptureOutputToMetadata` : Captures stdout into `Metadata["Cli:StdOut"]` .

2. `WebhookNotificationNode` (HTTP POST Webhook Notifier)

- **Purpose:** Sends JSON payloads via HTTP POST to external services (Discord, Slack, n8n, Zapier).
 - **Ports:**
 - *Inputs:* `In`
 - *Outputs:* `Out` , `Failed`
-

Category 8: Documents & PDFs (`FileFlow.Plugin.Documents`)

1. `PdfMergeNode` (PDF File Merging)

- **Purpose:** Combines multiple PDF documents into a single merged file using `PdfSharp`.
- **Ports:** `In` $\rightarrow$ `Out`, `MergedOut`.

2. `PdfSplitNode` (PDF Document Splitter)

- **Purpose:** Splits multi-page PDF documents into individual pages or custom ranges.
- **Ports:** `In` $\rightarrow$ `PageOut`, `Error`.

3. `PdfTextExtractorNode` (PDF Text Content Extractor)

- **Purpose:** Extracts structured text from PDF documents using `PdfPig` for search, classification, or content-based renaming.
- **Ports:** `In` $\rightarrow$ `Out`, `Error`.

4. `PdfMetadataNode` (PDF Metadata Inspector & Modifier)

- **Purpose:** Reads and updates standard document metadata (Title, Author, Subject, Keywords).
- **Ports:** `In` $\rightarrow$ `Out`.

Category 9: Network & Remote Storage (`FileFlow.Plugin.Network`)

1. `FtpUploadNode` (FTP / Secure FTPS Uploader)

- **Purpose:** Uploads files to FTP/FTPS servers with explicit or implicit TLS/SSL encryption and auto-retry.
- **Ports:** `In` $\rightarrow$ `Success`, `Failed`.

2. `SftpUploadNode` (SSH / SFTP Secure Transfer)

- **Purpose:** Securely transfers files to Linux servers / VPS over SSH File Transfer Protocol with password or private key (`.pem` / `.ppk`) authentication.
- **Ports:** `In` $\rightarrow$ `Success`, `Failed`.

3. `SmbCopyNode` (Local Network Shares & NAS UNC)

- **Purpose:** Copies files to Windows UNC network shares (`\\Server\Share`) or NAS storage with domain or local credentials.
- **Ports:** `In` $\rightarrow$ `Success`, `Failed`.

4. `WebDavUploadNode` (Nextcloud & ownCloud WebDAV Storage)

- **Purpose:** Uploads and syncs files with corporate WebDAV servers or personal cloud instances.

- **Ports:** In $\rightarrow$ Success , Failed .

5. RemoteDownloadNode (Remote HTTP / HTTPS / FTP File Downloader)

- **Purpose:** Downloads remote assets from the web or local intranet directly into the execution pipeline.
- **Ports:** In $\rightarrow$ Success , Failed .

Category 10: Data, Spreadsheets & Databases (FileFlow.Plugin.Data)

1. ExcelReaderNode (Excel Sheet Row Streamer)

- **Purpose:** Reads .xlsx / .xls spreadsheets in high-performance streaming with MiniExcel , emitting each row with columns injected into item.Metadata .
- **Ports:** In (optional) $\rightarrow$ RowOut .

2. CsvReaderNode (Delimited File Parser)

- **Purpose:** Reads CSV, TSV, and delimited text files with auto-detection of delimiters (, , ; , \t , |) and encoding options.
- **Ports:** In (optional) $\rightarrow$ RowOut .

3. DataLookupNode (VLOOKUP / Table Matching)

- **Purpose:** Looks up and enriches the current file against an external reference table (Excel, CSV, JSON) with instant $O(1)$ memory hashing.
- **Ports:** In $\rightarrow$ Matched , Unmatched .

4. ExcelReportGeneratorNode (Styled Excel Report Generator)

- **Purpose:** Collects metadata from processed files and generates a formatted .xlsx summary file upon workflow completion (OnWorkflowCompletedAsync).
- **Ports:** In $\rightarrow$ Out , Report .

5. CsvExportNode (CSV / TSV Exporter)

- **Purpose:** Exports and appends selected metadata fields into a delimited CSV file with customizable delimiters and append mode.
- **Ports:** In $\rightarrow$ Out .

6. SqliteDatabaseSinkNode (SQLite Audit Log & Storage)

- **Purpose:** Inserts execution audit records and traceability into a local SQLite database with automatic schema and index creation.
- **Ports:** In $\rightarrow$ Out .

7. `DataFormatConverterNode` (Data Format Converter)

- **Purpose:** Directly converts structured data files between `Excel (.xlsx)` $\rightleftharpoons$ `CSV` $\rightleftharpoons$ `JSON` .
 - **Ports:** `In` $\rightarrow$ `Out` .
-

Category 11: AI & Computer Vision (`FileFlow.Plugin.AI`)

1. `LocalOcrNode` (Optical Character Recognition OCR)

- **Purpose:** Extracts text and structured data from images and scanned documents into `{Ocr:Text}` , `{Ocr:WordCount}` , and `{Ocr:Language}` .
- **Ports:** `In` $\rightarrow$ `Out` , `Error` .

2. `SmartImageClassifierNode` (AI Photo Classifier)

- **Purpose:** Visually classifies photos into thematic categories (Landscapes, Invoices, Portraits, Vehicles, Food, etc.) in `{AI:Category}` , `{AI:TopLabel}` , and `{AI:Confidence}` .
- **Ports:** `In` $\rightarrow$ `Out` , `Error` .

3. `FaceDetectorNode` (Human Face Detector)

- **Purpose:** Detects human faces and branches the pipeline based on presence and count (`{AI:HasFaces}` and `{AI:FaceCount}`).
- **Ports:** `In` $\rightarrow$ `FacesFound` , `NoFaces` .

4. `ObjectDetectorNode` (YOLO Object Detector)

- **Purpose:** Detects and identifies multiple everyday objects (people, vehicles, pets, items) in `{AI:DetectedObjects}` and `{AI:TopObject}` .
- **Ports:** `In` $\rightarrow$ `Out` , `Error` .

5. `LocalWhisperTranscriberNode` (Whisper Speech-to-Text Transcriber)

- **Purpose:** Transcribes audio and video files to text and synchronized subtitles (`.srt`) in-process and privately into `{Transcript}` .
 - **Ports:** `In` $\rightarrow$ `Out` , `Error` .
-

6. Advanced Execution Modes & DAG Engine

Real-Time Watchdog Trigger Mode

- Toggle the  **Vigilante (Watchdog)** button in the top bar to listen continuously for incoming files in the configured source directories.

- Automatically processes new or modified files with intelligent debounce to prevent locked-file race conditions.

Performance Telemetry & Bottleneck Heatmap

- Microsecond-precision per-node latency measurements via `Stopwatch.GetTimestamp()`.
- Real-time visual latency badges ( 12 ms /  1.4 s) on node card headers with color-coded warning states ( Cuello de botella /  Bottleneck) for nodes consuming $>35\%$ of total execution time.

Headless CLI Runner (`fileflow.exe --run`)

Execute workflows unattended from PowerShell, bash, or Windows Task Scheduler:

```
# Standard run with dynamic variable injection
.\FileFlow.App.exe --run "workflow.json" --input "C:\Data" --var Environment=Production

# Override specific node parameters
.\FileFlow.App.exe --run "workflow.json" --param Throttle.DelayMilliseconds=50

# Continuous watch mode with structured JSON summary report
.\FileFlow.App.exe --run "workflow.json" --watch --summary "report.json"

# State checkpoint resumption
.\FileFlow.App.exe --run "workflow.json" --resume
```

State Checkpointing & Resumption

- Automatically persists progress in `%LocalAppData%\FileFlowStudio/checkpoints/`.
- If a long-running batch job is interrupted by a power failure or restart, re-running the workflow skips already-completed files (`CompletedFileKeys`).

7. Canvas Design & Productivity Tools

- **Sticky Notes:** Right-click canvas $\rightarrow$ `Create Sticky Note` to add comments, diagrams, or instructions with customizable colors and live resizing.
- **Group Frames (`Ctrl+G`):** Select multiple nodes and press `Ctrl+G` to encase them in an interactive group box. Moving the group by its title bar moves all contained nodes together.
- **Dynamic Category Dropdown:** Compact top bar dropdown with live search, favorite filters (), frequent usage badges (), and dynamic plugin category discovery.

8. Example Flows and Built-In Templates

Template 1: Automated Photo Sorting by Date & Camera

```
[FolderSourceNode]
  | (FileOut)
  ▼
[ExifMetadataNode]
  | (Out)
  ▼
[AdvancedRenamerNode (<Exif:Date:yyyyMMdd>_<Exif:CameraModel>_<Index:D3>.<Ext>)]
  | (Out)
  ▼
[FileRelocatorNode (Destination: {DestinationRoot}\{Year}\{Month})]
```

Template 2: Duplicate Cleaner to Windows Recycle Bin

```
[FolderSourceNode]
  | (FileOut)
  ▼
[HashCalculatorNode (SHA-256)]
  | (Out)
  ▼
[DeduplicationFilterNode]
  ├── (Unique)    → [LogOutputNode (Unique File Retained)]
  └── (Duplicate) → [OriginalFileActionNode (Action: MoveToRecycleBin)]
```

Template 3: Excel Lookup & Remote SFTP Dispatch

```
[FolderSourceNode (PDFs)]
  | (Out)
  ▼
[DataLookupNode (clients.xlsx by {FileNameWithoutExtension})]
  ├── (Matched)    → [SftpUploadNode (Client Server)] → [SqliteDatabaseSinkNode (Audit Log)]
  └── (Unmatched) → [FileRelocatorNode (Pending Review Folder)]
```